

Salyco — Deployment Guide

Complete, from-zero instructions for deploying the Salyco full-stack app (Django + PostgreSQL + React/Vite) to an Ubuntu VPS with automatic HTTPS.

- **Stack:** PostgreSQL 16 · Django + Gunicorn · React (Vite) served by Nginx · Caddy reverse proxy (HTTPS)
- **Everything runs in Docker.** You install Docker once; the app is 3 commands after that.
- **Domain:** `salyco.ir` (registered at irnic)
- **Target OS:** Ubuntu 22.04 / 24.04

Replace `YOUR_VPS_IP` with your server's IP and `salyco` with your chosen Linux username wherever they appear.

Table of contents

0. [Point your domain at the VPS \(DNS\)](#)
 1. [First login & secure the server](#)
 2. [Install Docker](#)
 3. [Get your code onto the server](#)
 4. [Configure environment \(`.env`\)](#)
 5. [Launch & verify](#)
 6. [Day-2 operations](#)
 7. [Troubleshooting](#)
 8. [Appendix: SSH tools & converting this doc](#)
-

Phase 0 — Point your domain at the VPS (DNS)

Do this first — DNS takes time to propagate while you do everything else.

Log into your **irnic** control panel and create these records for `salyco.ir`:

Type	Host / Name	Value
A	@	<code>YOUR_VPS_IP</code>
A	www	<code>YOUR_VPS_IP</code>

Save. Propagation takes anywhere from 5 minutes to a few hours.

Check from your Windows PC:

```
nslookup salyco.ir
```

When it returns `YOUR_VPS_IP`, DNS is ready. Continue with the other phases while you wait — HTTPS (Phase 5) is the only step that requires DNS to be live.

Phase 1 — First login & secure the server

1.1 Log in

From **Windows Terminal** (built-in `ssh`, no extra software needed):

```
ssh root@YOUR_VPS_IP
```

Enter the password your host provided.

1.2 Update the system

```
apt update && apt upgrade -y
```

1.3 Create a non-root user

Never run day-to-day operations as `root`.

```
adduser salyco          # set a password when prompted
usermod -aG sudo salyco # grant admin (sudo) rights
```

1.4 Set up passwordless SSH key login

On your Windows PC (open a new terminal — don't close the server one yet), generate a key if you don't already have one:

```
ssh-keygen -t ed25519
# Press Enter through the prompts. A passphrase is optional but recommended.
```

Copy the public key to the server's new user:

```
type $env:USERPROFILE\.ssh\id_ed25519.pub | ssh salyco@YOUR_VPS_IP "mkdir -p
~/.ssh && cat >> ~/.ssh/authorized_keys && chmod 700 ~/.ssh && chmod 600
~/.ssh/authorized_keys"
```

Test it — this should log in **without** asking for a password:

```
ssh salyco@YOUR_VPS_IP
```

From now on, log in as **salyco**, not **root**.

1.5 Firewall

Allow only SSH, HTTP, and HTTPS:

```
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable          # type 'y' to confirm
sudo ufw status          # verify the rules
```

Phase 2 — Install Docker

```
# Docker's official install script
curl -fsSL https://get.docker.com | sudo sh

# Allow your user to run docker without sudo
sudo usermod -aG docker $USER

# Log out and back in so the group change takes effect
exit
```

Log back in (`ssh salyco@YOUR_VPS_IP`), then verify:

```
docker --version
docker compose version
```

Both should print a version number.

Phase 3 — Get your code onto the server

Two options. **Git (Way A) is recommended** — updates become a one-line command and it automatically skips files you don't want on the server.

Way A — Git (recommended)

One-time, on your PC

Make sure secrets and junk aren't committed. Your **.gitignore** already excludes **.env**, **node_modules/**, ***.sqlite3**, and **Backend/env/**. Remove files that were tracked *before* **.gitignore** existed (this keeps your local copies, only stops tracking them):

```
git rm -r --cached Backend/db.sqlite3
git rm -r --cached $(git ls-files "*.pyc")
git add -A
git commit -m "Untrack cache/db files; deployment config"
git push -u origin master
```

If the repo isn't on a Git host yet: create an **empty private repo** on GitHub, then `git remote add origin https://github.com/YOUR_USER/salyco.git` before the `git push` above.

On the server

```
cd ~
git clone https://github.com/YOUR_USER/salyco.git salyco
cd salyco
```

For a **private repo**, GitHub asks for a username + **personal access token** (not your account password). Create one at: GitHub → Settings → Developer settings → Personal access tokens → Fine-grained token with read access to the repo. Paste it as the password.

Way B — Direct upload (SFTP / SCP, no Git)

Only if you don't want a Git host. **Never upload `node_modules`, `Backend/env`, `Backend/venv`, or `__pycache__`** — they're huge, pointless (rebuilt in Docker), and can take hours.

Option B1 — robocopy + scp (built into Windows). From `E:\salyco-fullstack`:

```
# Make a clean staging copy without the junk
robocopy . ..\salyco-deploy /E /XD node_modules env venv __pycache__ staticfiles
.git /XF db.sqlite3 *.pyc

# Upload the clean copy
scp -r ..\salyco-deploy salyco@YOUR_VPS_IP:~/salyco
```

Option B2 — Termius / Bitvise SFTP panel. Delete the folders listed above from a *copy* of your project first, then drag the folder into `/home/salyco/`.

After upload (either option)

```
cd ~/salyco
ls # confirm docker-compose.yml, Backend/, Frontend/, Caddyfile are present
```

Updating later

Method	Update command
Git	<code>git pull</code> on the server, then rebuild (see Phase 6)
Upload	Re-run the robocopy + scp / drag-and-drop each time

Phase 4 — Configure environment (`.env`)

The stack reads all secrets and domain settings from a `.env` file that lives **only on the server** (it's git-ignored, never committed).

4.1 Create it from the template

```
cd ~/salyco
cp .env.example .env
```

4.2 Generate a strong Django secret key

```
docker run --rm python:3.12-slim python -c "import secrets;
print(secrets.token_urlsafe(50))"
```

Copy the output — you'll paste it as `SECRET_KEY`.

4.3 Edit the file

```
nano .env
```

Fill in real values:

```
# Django
SECRET_KEY=<paste the generated key from step 4.2>
DEBUG=False
ALLOWED_HOSTS=salyco.ir,www.salyco.ir,localhost
CSRF_TRUSTED_ORIGINS=https://salyco.ir,https://www.salyco.ir
FRONTEND_BASE_URL=https://salyco.ir

# Postgres
POSTGRES_DB=salyco
POSTGRES_USER=salyco
POSTGRES_PASSWORD=<a long, strong, random password>
```

Save in nano: `Ctrl+O`, `Enter`, then `Ctrl+X`.

Security notes

- `DEBUG=False` is mandatory in production — never leave it `True`.
- The `SECRET_KEY`, `POSTGRES_PASSWORD`, and every value above are pulled from this `.env` by `docker-compose.yml`. Nothing is hardcoded.
- This file contains live credentials. Keep it off Git and off shared drives.

Phase 5 — Launch & verify

Requires DNS from **Phase 0** to be pointing at this server, so Caddy can obtain the HTTPS certificate. Confirm with `nslookup salyco.ir` first.

5.1 Build and start everything

```
cd ~/salyco
docker compose up -d --build
```

First build takes a few minutes (installing Python deps, building the React bundle). The `-d` runs it in the background.

5.2 Watch it come up

```
docker compose logs -f
```

Look for:

- `db` → "database system is ready to accept connections"
- `backend` → migrations applied, then Gunicorn "Booting worker"
- `caddy` → "certificate obtained successfully" for `salyco.ir`

Press `Ctrl+C` to stop following logs (the containers keep running).

5.3 Confirm all containers are healthy

```
docker compose ps
```

All four services (`db`, `backend`, `frontend`, `caddy`) should show `Up`.

5.4 Create your admin account

```
docker compose exec backend python manage.py createsuperuser
```

5.5 Test in the browser

- **Site:** <https://salyco.ir> (padlock icon = HTTPS working)
- **Admin:** <https://salyco.ir/admin/>
- **API:** <https://salyco.ir/api/>

You're live. 🎉

Phase 6 — Day-2 operations (updates, backups, logs)

Deploy a code update

Git workflow (Way A):

```
cd ~/salyco
git pull
docker compose up -d --build
```

Migrations run automatically on backend startup (via `entrypoint.sh`).

Common commands

```
# View logs (all services, or one)
docker compose logs -f
docker compose logs -f backend

# Restart / stop / start
docker compose restart backend
docker compose down          # stop & remove containers (data volumes are kept)
docker compose up -d         # start again

# Run a Django management command
docker compose exec backend python manage.py <command>

# Open a shell inside the backend container
docker compose exec backend bash
```

Back up the database

```
# Dump Postgres to a timestamped file on the host
docker compose exec -T db pg_dump -U salyco salyco > ~/salyco-backup-$(date +%F).sql
```

Restore into a fresh database:

```
cat ~/salyco-backup-YYYY-MM-DD.sql | docker compose exec -T db psql -U salyco -d salyco
```

Store backups off the server too (download with `scp` or your SFTP client). Consider a weekly cron job for automated dumps.

Where persistent data lives

Docker named volumes survive `docker compose down` and rebuilds:

Volume	Contents
<code>postgres_data</code>	The PostgreSQL database
<code>media_data</code>	User uploads (product/article images)
<code>static_data</code>	Collected Django/admin static files
<code>caddy_data</code>	HTTPS certificates

Do not run `docker compose down -v` unless you intend to **delete all data** — the `-v` flag wipes these volumes.

Troubleshooting

Caddy can't get a certificate / HTTPS fails

- DNS isn't propagated yet: `nslookup salyco.ir` must return your VPS IP.
- Ports 80/443 blocked: confirm `sudo ufw status` allows them and your VPS provider's firewall/security-group also allows them.
- Check logs: `docker compose logs caddy`

502 Bad Gateway

- Backend crashed or still starting. `docker compose logs backend`.
- Common cause: a wrong value in `.env` (e.g. bad DB password). Fix `.env`, then `docker compose up -d`.

Backend can't connect to the database

- `POSTGRES_*` values in `.env` must match between the `db` and `backend` services (they both read the same `.env`, so just fix `.env`).
- If you changed the password *after* the first run, the volume still has the old one. Either restore the old password or reset (destroys data): `docker compose down -v && docker compose up -d --build`.

Static files or admin CSS missing

- `collectstatic` runs on backend startup; check `docker compose logs backend`.

Site loads but API calls fail with 403 CSRF

- Ensure `CSRF_TRUSTED_ORIGINS` in `.env` includes `https://salyco.ir`.

Changed `.env` but nothing changed

- Re-run `docker compose up -d` (recreates containers with new env values).
-

Appendix

SSH tool recommendation

You're on Windows 11, which ships with OpenSSH. Options, best to simplest:

- **Windows Terminal + built-in `ssh`** — nothing to install; ideal for commands.
- **Termius** (free) — saved hosts, synced keys, and a drag-drop SFTP panel.
- **Bitvise** — fine; its strength is the visual SFTP pane (see Way B, Option B2).

Any of these work. The commands in this guide assume the built-in `ssh`.

Converting this document to PDF or HTML

This is a Markdown file. To produce a shareable PDF or HTML:

- **VS Code:** install the "Markdown PDF" extension, right-click the file → "Markdown PDF: Export (pdf)".
 - **Pandoc** (if installed): `pandoc DEPLOYMENT.md -o DEPLOYMENT.pdf`
 - **GitHub:** just view the file in your repo — it renders as a formatted page.
-

Architecture reference: `docker-compose.yml` (service definitions), `Caddyfile` (HTTPS + domains), `Backend/Dockerfile` + `Backend/entrypoint.sh` (Django/Gunicorn, auto-migrations), `Frontend/salyco-front/Dockerfile` + `nginx.conf` (React build + reverse proxy to Django).